PRUEBA TECNICA DESARROLLADOR

Desarrollador Fullstack (Angular + Django)

Fecha de inicio: 02-Mar-2026 10:00
Fecha de entrega: 06-Mar-2026 10:00
duración estimada: 60 horas

CONTEXTO DEL NEGOCIO

DataPulse Latam es una firma de consultoría financiera que asesora a fondos de inversión sobre mercados latinoamericanos. Actualmente, su equipo de analistas dedica gran parte de su jornada a:

- Recopilar manualmente indicadores económicos (PIB, inflación, desempleo) de múltiples fuentes
- Crear portafolios de inversión simulados en hojas de calculo
- Calcular índices de riesgo país de forma manual combinando variables económicas, cambiarias y sociales
- Generar reportes en Word/PDF copiando datos de distintas fuentes
- Monitorear tipos de cambio y sus variaciones para tomar decisiones de cobertura

La empresa necesita una **plataforma web completa** que centralice estas operaciones, permita a múltiples analistas trabajar simultáneamente y ofrezca visualizaciones interactivas para la toma de decisiones.

OBJETIVO DE LA PRUEBA

Desarrollar una **aplicación web fullstack** compuesta por:

1. **Backend API REST** con Django + Django REST Framework
2. **Frontend SPA** con Angular 12+ y Angular Material
3. **Base de datos relacional** con modelo de datos normalizado
4. **Consumo de APIs externas** para datos económicos en tiempo real
5. **Despliegue en nube** con la aplicación funcional y accesible vía URL publica

---

REQUERIMIENTOS TECNICOS

PARTE 1: Diseño de Base de Datos y Backend API (40% de la evaluación)

1.1 Modelo de Datos

Diseñar e implementar el siguiente modelo relacional en la base de datos. El candidato debe crear las migraciones de Django correspondientes.

Entidades requeridas:

Entidad: Usuario (extiende AbstractUser de Django)

- email (unico, usado como login)
- nombre_completo
- rol (ADMIN | ANALISTA | VIEWER)
- fecha_creacion
- activo

Entidad: País

- codigo_iso (PK, ej: "CO", "BR", "MX")
- nombre
- moneda_codigo (ej: "COP", "BRL", "MXN")
- moneda_nombre
- region (ANDINA | CONO_SUR | CENTROAMERICA | CARIBE)
- latitud
- longitud
- poblacion
- activo

Entidad: IndicadorEconomico

- pais (FK -> Pais)
- tipo (PIB | INFLACION | DESEMPLEO | BALANZA_COMERCIAL | DEUDA_PIB | PIB_PERCAPITA)
- valor
- unidad (PORCENTAJE | USD | USD_MILES_MILLONES)
- anio
- fuente (WORLD_BANK | MANUAL)
- fecha_actualizacion

Entidad: TipoCambio

- moneda_origen (FK -> Pais.moneda_codigo)
- moneda_destino (default: "USD")

- tasa
- fecha
- variacion_porcentual (respecto al dia anterior)
- fuente

Entidad: Portafolio

- nombre
- descripcion
- usuario (FK -> Usuario, creador)
- fecha_creacion
- fecha_modificacion
- activo
- es_publico (boolean, visible para otros usuarios)

Entidad: Posicion (inversion dentro de un portafolio)

- portafolio (FK -> Portafolio)
- pais (FK -> Pais)
- tipo_activo (RENTA_FIJA | RENTA_VARIABLE | COMMODITIES | MONEDA)
- monto_inversion_usd
- fecha_entrada
- fecha_salida (nullable)
- notas

Entidad: IndiceRiesgo (calculado por el sistema)

- pais (FK -> Pais)
- fecha_calculo
- score_economico (0-100)
- score_cambiario (0-100)
- score_estabilidad (0-100)
- indice_compuesto (0-100)
- nivel_riesgo (BAJO | MODERADO | ALTO | CRITICO)
- detalle_calculo (JSONField con el desglose)

Entidad: Alerta

- usuario (FK -> Usuario, nullable para alertas globales)
- pais (FK -> Pais)
- tipo_alerta (RIESGO | TIPO_CAMBIO | INDICADOR)
- severidad (INFO | WARNING | CRITICAL)
- titulo
- mensaje

- `leida (boolean)`
- `fecha_creacion`

Entidad: LogActividad

- `usuario (FK -> Usuario)`
- `accion (CREAR | EDITAR | ELIMINAR | CONSULTAR | LOGIN | EXPORT)`
- `entidad_afectada (nombre del modelo)`
- `entidad_id`
- `detalle (JSONField)`
- `ip_address`
- `fecha`

Restricciones de integridad que deben implementarse:

- Un usuario no puede tener dos portafolios con el mismo nombre
- No puede haber dos registros de IndicadorEconomico para el mismo pais + tipo + año
- Las posiciones con fecha_salida deben tener fecha_salida > fecha_entrada
- El indice_compuesto en IndiceRiesgo debe estar entre 0 y 100
- Un Viewer no puede crear ni editar portafolios

1.2 API REST - Endpoints Requeridos

Implementar los siguientes endpoints con Django REST Framework:

Autenticación (JWT):

Método	Endpoint	Descripción
POST	`/api/auth/register/`	Registro de usuario
POST	`/api/auth/login/`	Login (retorna access + refresh token)
POST	`/api/auth/refresh/`	Refresh token
GET	`/api/auth/me/`	Perfil del usuario autenticado
PUT	`/api/auth/me/`	Actualizar perfil

Paises e Indicadores:

Método	Endpoint	Descripción
GET	`/api/paises/`	Listar paises (con filtros por region)
GET	`/api/paises/{codigo_iso}/`	Detalle de pais con indicadores
GET	`/api/paises/{codigo_iso}/indicadores/`	Indicadores de un pais (filtro por tipo, anio)
GET	`/api/paises/{codigo_iso}/tipo-cambio/`	Historico tipo de cambio (filtro por rango de fechas)
POST	`/api/paises/sync-indicadores/`	Disparar sincronizacion desde APIs externas (solo ADMIN)

Portafolios:

Método	Endpoint	Descripción
GET	`/api/portafolios/`	Listar portafolios del usuario + publicos

POST	/api/portafolios/	Crear portafolio
GET	/api/portafolios/{id}/	Detalle con posiciones y metricas
PUT	/api/portafolios/{id}/	Actualizar portafolio
DELETE	/api/portafolios/{id}/	Eliminar (soft delete)
POST	/api/portafolios/{id}/posiciones/	Agregar posicion
PUT	/api/portafolios/{id}/posiciones/{id}/	Editar posicion
DELETE	/api/portafolios/{id}/posiciones/{id}/	Cerrar posicion (set fecha_salida)
GET	/api/portafolios/{id}/resumen/	Resumen: distribucion por pais, tipo_activo, riesgo
GET	/api/portafolios/{id}/export/pdf/	Exportar reporte PDF del portafolio

índice de Riesgo:

Método	Endpoint	Descripción
GET	/api/riesgo/	Ranking de paises por indice de riesgo
GET	/api/riesgo/{codigo_iso}/	Detalle de riesgo de un pais con historico
GET	/api/riesgo/{codigo_iso}/historico/	Historico de indices (filtro por rango)
POST	/api/riesgo/calcular/	Recalcular indices para todos los paises (solo ADMIN)

Alertas:

Método	Endpoint	Descripción
GET	/api/alertas/	Alertas del usuario (filtro: leidas/no leidas, tipo)
PUT	/api/alertas/{id}/leer/	Marcar como leida
PUT	/api/alertas/leer-todas/	Marcar todas como leidas
GET	/api/alertas/resumen/	Conteo por tipo y severidad

Dashboard:

Método	Endpoint	Descripción
GET	/api/dashboard/resumen/	KPIs globales (total paises, alertas activas, portafolios)
GET	/api/dashboard/mapa/	Datos para mapa: paises con coordenadas + indice riesgo + color
GET	/api/dashboard/tendencias/	Datos de tendencia para graficos (indicadores ultimos 5 anios)

Requisitos transversales de la API:

- Paginación en todos los endpoints de listado (page_size configurable, default 20)
- Filtros por query params donde se especifica
- Ordenamiento (ordering) en listados
- Búsqueda por texto (search) en países y portafolios
- Respuestas de error estandarizadas con formato consistente
- Throttling: 100 req/min para usuarios autenticados, 20 req/min para anónimos
- Permisos basados en rol (ADMIN, ANALISTA, VIEWER)
- Validaciones de negocio con mensajes de error claros en español

1.3 Fuentes de Datos Externas

Consumir las siguientes APIs públicas para poblar la base de datos:

A. World Bank API - Indicadores económicos

- **Documentación:** https://datahelpdesk.worldbank.org/knowledgebase/articles/889392
- **URL Base:** https://api.worldbank.org/v2/

Indicador	Código World Bank	Tipo en BD
PIB (USD corrientes)	NY.GDP.MKTP.CD	PIB
inflación (% anual)	FP.CPI.TOTL.ZG	INFLACION
Desempleo (% total)	SL.UEM.TOTL.ZS	DESEMPLEO
Balanza comercial (% PIB)	NE.RSB.GNFS.ZS	BALANZA_COMERCIAL
Deuda gobierno (% PIB)	GC.DOD.TOTL.GD.ZS	DEUDA_PIB
PIB per cápita (USD)	NY.GDP.PCAP.CD	PIB_PERCAPITA

Ejemplo de llamada (PIB de Colombia, últimos 5 años):

```
https://api.worldbank.org/v2/country/CO/indicator/NY.GDP.MKTP.CD?date=2019:2023&format=json
```

Países para consultar: CO, BR, MX, AR, CL, PE, EC, UY, PY, PA

B. ExchangeRate-API - Tipos de Cambio

- **Documentación:** https://www.exchangerate-api.com/docs/free
- **URL Base:** https://api.exchangerate-api.com/v4/latest/USD
- Tasa actual USD vs monedas locales de los 10 países
- Variación diaria (calcular almacenando datos históricos)

C. REST Countries API - Datos de Países

- **URL Base:** https://restcountries.com/v3.1/
- **Ejemplo:** https://restcountries.com/v3.1/alpha/CO
- Nombre oficial, moneda, población, coordenadas, bandera (URL)

1.4 Reglas de Negocio - Indice de Riesgo País Compuesto (IRPC)

El sistema debe calcular automáticamente un índice de riesgo para cada país siguiendo esta fórmula:

```
IRPC = (Score_Economico * 0.40) + (Score_Cambiario * 0.30) + (Score_Estabilidad * 0.30)
```

Cálculo del Score Económico (0-100):

```
def calcular_score_economico(pais):
    score = 100

    # PIB per capita (penalizar si es bajo)
    pib_pc = obtener_ultimo_indicador(pais, 'PIB_PERCAPITA')
    if pib_pc < 3000:    score -= 30
```

```
    elif pib_pc < 6000:  score -= 15
    elif pib_pc < 12000: score -= 5

    # Inflacion (penalizar si es alta)
    inflacion = obtener_ultimo_indicador(pais, 'INFLACION')
    if inflacion > 50:   score -= 40
    elif inflacion > 10: score -= 25
    elif inflacion > 5:  score -= 10

    # Desempleo (penalizar si es alto)
    desempleo = obtener_ultimo_indicador(pais, 'DESEMPLEO')
    if desempleo > 15:   score -= 25
    elif desempleo > 10: score -= 15
    elif desempleo > 7:  score -= 5

    # Deuda/PIB (penalizar si es alta)
    deuda = obtener_ultimo_indicador(pais, 'DEUDA_PIB')
    if deuda > 80:       score -= 20
    elif deuda > 50:     score -= 10

    return max(0, score)
```

Cálculo del Score Cambiario (0-100):

```
def calcular_score_cambiario(pais):
    score = 100

    # Volatilidad (desviacion estandar de variaciones diarias, 30 dias)
    variaciones = obtener_variaciones_tipo_cambio(pais, dias=30)
    volatilidad = desviacion_estandar(variaciones)
    if volatilidad > 3.0:    score -= 40    # Extremadamente volatil
    elif volatilidad > 1.5:  score -= 25    # Muy volatil
    elif volatilidad > 0.5:  score -= 10    # Moderadamente volatil

    # Tendencia (depreciacion acumulada ultimos 30 dias)
    depreciacion = calcular_depreciacion(variaciones)
    if depreciacion > 10:    score -= 30
    elif depreciacion > 5:   score -= 15
    elif depreciacion > 2:   score -= 5

    return max(0, score)
```

Cálculo del Score de Estabilidad (0-100):

```
def calcular_score_estabilidad(pais):
    score = 100

    # Balanza comercial
    balanza = obtener_ultimo_indicador(pais, 'BALANZA_COMERCIAL')
    if balanza < -10:    score -= 25    # Deficit severo
    elif balanza < -5:   score -= 15    # Deficit moderado
    elif balanza < 0:    score -= 5     # Deficit leve
```

```
    # Tendencia del PIB (comparar ultimo anio vs anterior)
    pib_actual = obtener_indicador(pais, 'PIB', anio_actual)
    pib_anterior = obtener_indicador(pais, 'PIB', anio_actual - 1)
    if pib_anterior and pib_actual:
        crecimiento = ((pib_actual - pib_anterior) / pib_anterior) * 100
        if crecimiento < -2:   score -= 30    # Recesion
        elif crecimiento < 0:  score -= 20    # Contraccion
        elif crecimiento < 1:  score -= 10    # Estancamiento

    # Concentracion de indicadores negativos
    indicadores_negativos = contar_indicadores_en_riesgo(pais)
    score -= (indicadores_negativos * 5)

    return max(0, score)
```

Sistema de Clasificación del IRPC:

IRPC Score	Nivel de Riesgo	Color	Hex Code	Acción Recomendada
75-100	BAJO	**Verde**	#22c55e	Favorable para inversión
50-74	MODERADO	**Amarillo**	#eab308	Invertir con precaución
25-49	ALTO	**Naranja**	#f97316	Reducir exposición
0-24	CRITICO	**Rojo**	#ef4444	Evitar nueva inversión

Ejemplo de Calculo para Colombia (CO):

Datos de entrada (hipotéticos):

- PIB per cápita: $6,800 USD -> penalización -5
- inflación: 9.2% -> penalización -25
- Desempleo: 11.3% -> penalización -15
- Deuda/PIB: 55% -> penalización -10
- Volatilidad cambiaria (std): 0.8% -> penalización -10
- Depreciación acumulada 30d: 1.5% -> penalización 0
- Balanza comercial: -4.2% -> penalización -5
- Crecimiento PIB: 1.5% -> penalización 0
- Indicadores negativos: 3 -> penalización -15

Calculo:

```
Score Economico = 100 - 5 - 25 - 15 - 10 = 45
Score Cambiario = 100 - 10 - 0 = 90
Score Estabilidad = 100 - 5 - 0 - 15 = 80

IRPC = (45 * 0.40) + (90 * 0.30) + (80 * 0.30)
IRPC = 18 + 27 + 24 = 69
```

```
Resultado: IRPC = 69 (Riesgo MODERADO - amarillo)
Accion: Invertir con precaucion
```

Generación automática de alertas basada en IRPC:

El sistema debe generar alertas automáticamente cuando:

6. **CRITICAL:** IRPC cae por debajo de 25 para cualquier país
7. **WARNING:** IRPC cae más de 15 puntos respecto al cálculo anterior
8. **WARNING:** Variación tipo de cambio > 3% en un día
9. **INFO:** Nuevos datos económicos disponibles tras sincronización
10. **CRITICAL:** inflación supera 50% (hiperinflación)

PARTE 2: Frontend Angular (35% de la evaluación)

2.1 Estructura de la Aplicación

Desarrollar una SPA en Angular 12+ con la siguiente estructura de módulos:

```
src/app/
├── core/
│   ├── auth/
│   │   ├── services/auth.service.ts       - Login, registro, token management
│   │   ├── guards/auth.guard.ts           - Proteger rutas autenticadas
│   │   ├── guards/role.guard.ts           - Proteger rutas por rol
│   │   └── interceptors/jwt.interceptor.ts- Adjuntar token a requests
│   ├── services/
│   │   ├── api.service.ts                 - HTTP client wrapper con error handling
│   │   ├── pais.service.ts               - Operaciones de paises
│   │   ├── portafolio.service.ts         - Operaciones de portafolios
│   │   ├── riesgo.service.ts             - Operaciones de riesgo
│   │   └── alerta.service.ts             - Operaciones de alertas
│   └── interceptors/
│       ├── error.interceptor.ts           - Manejo global de errores HTTP
│       └── loading.interceptor.ts         - Indicador de carga global
├── modules/
│   ├── auth/         (login/, register/)
│   ├── dashboard/    (mapa-riesgo/, kpi-cards/, tabla-ranking/, grafico-tendencias/)
│   ├── paises/       (lista-paises/, detalle-pais/)
│   ├── portafolios/  (lista/, crear/, detalle/, posicion-form/, distribucion-chart/)
│   └── alertas/      (panel-alertas/, alerta-badge/)
└── shared/           (loading-spinner/, confirm-dialog/, pipes/, directives/)
```

2.2 Vistas Requeridas

A. Login y Registro

- Formulario reactivo con validaciones

- Manejo de errores de autenticación (credenciales invalidas, usuario ya existe)
- Redirección post-login al dashboard
- Persistencia de sesión (token en localStorage)

B. Dashboard Principal

- Mapa de Latinoamérica mostrando cada país con indicador de color según IRPC
- 4 tarjetas KPI: Total países monitoreados, Alertas activas, Portafolios del usuario, Promedio IRPC región
- Tabla ranking: Los 10 países ordenados por IRPC con columnas: País, IRPC, Nivel, Variación, Tendencia
- Gráfico de tendencias: Selector de indicador + grafico de líneas comparando 3 países seleccionables, últimos 5 años

C. Detalle de País

- Información general (bandera, nombre, moneda, región, población)
- Indicadores económicos en tarjetas con valores actuales
- Gráfico histórico de indicadores (5 años)
- histórico de tipo de cambio (grafico de línea, últimos 30 días)
- índice de riesgo actual con desglose visual (3 scores + compuesto)
- Tabla de histórico de IRPC

D. Gestión de Portafolios

- Listado de portafolios propios y públicos (con badge diferenciador)
- Crear/editar portafolio con formulario reactivo
- Lista de posiciones en tabla editable
- Formulario para agregar posición con selectores (país, tipo_activo, monto)
- Gráfico pie: distribución por país + Grafico donut: distribución por tipo_activo
- Calculo en tiempo real: monto total invertido, riesgo promedio ponderado
- botón exportar a PDF
- Confirmación antes de eliminar (dialog)

E. Panel de Alertas

- Lista de alertas con filtros (tipo, severidad, leídas/no leídas)
- Marcar como leída individual y masivamente
- Badge en navbar con conteo de no leídas (polling cada 30 seg)
- Iconos y colores diferenciados por severidad

2.3 Requisitos Técnicos del Frontend

- **Angular 12+** con strict mode
- **Angular Material** para componentes UI
- **Formularios reactivos** (ReactiveFormsModule) para todos los formularios
- **Lazy loading** de módulos de feature (dashboard, paises, portafolios, alertas)
- **Route guards** (AuthGuard, RoleGuard)
- **HTTP Interceptors** (JWT, Error handling, Loading)
- **Manejo de estado:** Implementar servicio de estado para al menos un feature usando BehaviorSubject/Observable o NgRx
- **Responsive design:** desktop (1920px) y tablet (768px)
- **Gráficos:** Usar ngx-charts, Chart.js, o Plotly para las visualizaciones
- **Mapa:** Implementar visualización geográfica de Latam con indicadores de riesgo

2.4 Formularios y Validaciones del Frontend

Formulario de Portafolio:

- Nombre: requerido, min 3 caracteres, max 100, único (validación async contra API)
- Descripción: max 500 caracteres, con contador de caracteres visible
- Es público: checkbox

Formulario de Posicion:

- País: requerido, selector con búsqueda (autocomplete)
- Tipo activo: requerido, selector
- Monto inversión USD: requerido, numérico, min $1,000, max $10,000,000, formato con separadores de miles
- Fecha entrada: requerido, no puede ser futura
- Notas: opcional, max 200 caracteres

Validaciones cruzadas:

- No se puede agregar una posición de tipo MONEDA para un país cuya moneda es USD
- El monto total del portafolio no puede superar $50,000,000 USD (validar al agregar posición)
- No se pueden tener más de 2 posiciones activas del mismo tipo_activo en el mismo país

---

PARTE 3: Integración, Tareas Asincronas y Despliegue (25% de la evaluación)

3.1 Tareas en Background (Celery o Django Management Commands)

Implementar al menos dos tareas automatizadas:

Tarea 1: Sincronización de Indicadores económicos

- Ejecutar bajo demanda (endpoint POST solo ADMIN) o programada
- Consultar World Bank API para los 10 países y 6 indicadores
- Manejar errores: si la API falla para un país, continuar con los demás y registrar error
- Registrar en logs: inicio, fin, países procesados, errores encontrados
- Generar alerta INFO indicando que se actualizaron los datos

Tarea 2: Recalculo de índice de Riesgo

- Ejecutar después de cada sincronización de indicadores
- Calcular IRPC para los 10 países según las fórmulas de la Sección 1.4
- Comparar con calculo anterior y generar alertas si hay cambios significativos
- Almacenar resultado con detalle completo del calculo

Tarea 3: Actualización de Tipos de Cambio

- Consultar ExchangeRate-API
- Almacenar tasa diaria para cada moneda
- Calcular variación respecto al día anterior
- Generar alerta si variación > 3%

3.2 Manejo de Errores y Logs

Backend:

- Logging estructurado con Python logging (nivel INFO para operaciones normales, ERROR para fallos)
- Registro de actividad en tabla LogActividad para acciones CRUD
- Middleware personalizado para logging de requests (metodo, path, usuario, duración)
- Manejo de excepciones personalizado en DRF (custom exception handler)

Frontend:

- Interceptor de errores HTTP que muestre snackbar con mensaje descriptivo
- Manejo de errores de red (sin conexión)
- Redirección a login cuando el token expira (error 401)
- Página 404 para rutas no encontradas

3.3 Despliegue en Nube

Desplegar la aplicación completa en un servicio de nube gratuito. Opciones sugeridas:

Componente	Opciones Gratuitas
Backend Django	Railway, Render, PythonAnywhere, Fly.io
Frontend Angular	Vercel, Netlify, GitHub Pages, Firebase Hosting
Base de Datos	PostgreSQL en Railway/Render, SQLite para simplicidad
Redis (si usa Celery)	Redis en Railway, Upstash

Requisitos del despliegue:

- URL publica funcional para el backend (API accesible)
- URL publica funcional para el frontend (SPA accesible)
- CORS configurado correctamente entre frontend y backend
- Variables de entorno para configuración sensible (no hardcodear secrets)
- Base de datos con datos precargados (fixtures o seed script)
- HTTPS obligatorio

Datos precargados (seed data):

- 10 países latinoamericanos con datos completos
- Al menos 3 años de indicadores económicos por país
- 30 días de histórico de tipo de cambio
- 1 usuario ADMIN (admin@datapulse.com / DataPulse2026!)
- 1 usuario ANALISTA (analista@datapulse.com / DataPulse2026!)
- 1 usuario VIEWER (viewer@datapulse.com / DataPulse2026!)
- 2 portafolios de ejemplo con posiciones
- Índices de riesgo calculados para todos los países

ENTREGABLES

1. Código Fuente (50%)

Repositorio GitHub público con:

- Código del backend Django en carpeta /backend
- Código del frontend Angular en carpeta /frontend
- Archivo docker-compose.yml para levantar todo el stack local (opcional pero suma puntos)
- Archivo .env.example con todas las variables de entorno necesarias
- Commits incrementales que muestren la evolución del desarrollo (NO un solo commit final)
- .gitignore apropiado (sin node_modules, __pycache__, .env, db.sqlite3, etc.)

2. Documentación Técnica (20%)

README.md que incluya:

- **Arquitectura de la solución** con diagrama (puede ser ASCII, draw.io, o imagen)
- **Diagrama Entidad-Relación** de la base de datos
- **Decisiones técnicas:** porque se eligió cada librería/herramienta y alternativas consideradas
- **Instrucciones de instalación y ejecución local** paso a paso (deben ser reproducibles)

- **Instrucciones para ejecutar tests**
- **Endpoints de la API** documentados (puede usar link a Swagger/Redoc)
- **Manejo de errores** implementado y como se registran
- **URLs de despliegue** (frontend y backend)
- **Credenciales de prueba** para acceder a la aplicación desplegada

3. Aplicación Desplegada (20%)

- URL funcional del frontend
- URL funcional del backend (con acceso a /api/ o Swagger)
- Datos precargados listos para probar
- Los 3 usuarios de prueba funcionales

4. Video Demostrativo (10%)

Video de **5-8 minutos** mostrando:

- Login y navegación general
- Dashboard con mapa y gráficos funcionando
- Crear un portafolio y agregar posiciones
- Ver detalle de un país con indicadores y riesgo
- Panel de alertas
- Ejecutar sincronización de datos (si es ADMIN)
- Explicación breve de una decisión de arquitectura

ESCENARIOS DE PRUEBA

La solución debe manejar correctamente estos casos:

11. 1. **API externa no disponible:** Si World Bank o ExchangeRate-API no responden, la sincronización debe continuar con los datos que pueda obtener y registrar los errores.
12. 2. **Token expirado:** Si el JWT expira mientras el usuario navega, el frontend debe redirigir al login automáticamente y mostrar mensaje apropiado.
13. 3. **Permisos insuficientes:** Si un VIEWER intenta crear un portafolio, la API debe retornar 403 y el frontend debe ocultar los botones de creación/edición.
14. 4. **Datos incompletos:** Si un país no tiene todos los indicadores, el cálculo de IRPC debe funcionar con los datos disponibles y marcar los faltantes.
15. 5. **Concurrencia:** Si dos usuarios editan el mismo portafolio publico simultáneamente, debe manejarse con mensajes apropiados.
16. 6. **Validaciones de formulario:** Intentar enviar formularios con datos inválidos debe mostrar mensajes de error específicos sin hacer request al backend.

CRITERIOS DE EVALUACION

Arquitectura y Diseño de BD (20%)

- Modelo de datos normalizado y bien relacionado
- Migraciones de Django correctas y ordenadas
- Restricciones de integridad implementadas (validators, constraints)
- Índices en campos de búsqueda/filtro frecuente

Backend API (25%)

- Todos los endpoints funcionan correctamente
- Autenticación JWT implementada
- Permisos por rol funcionando
- Paginación, filtros y búsqueda implementados
- Serializadores con validaciones adecuadas
- Cálculo de IRPC correcto según formula
- Consumo de APIs externas con manejo de errores
- Logging y registro de actividad

Frontend Angular (25%)

- Estructura modular con lazy loading
- Formularios reactivos con validaciones
- Guards e interceptors funcionando
- gráficos y mapa interactivos
- Responsive (desktop + tablet)
- Manejo de estado para al menos un feature
- UX coherente: loading states, empty states, error messages

Integración y Despliegue (15%)

- Frontend y backend comunicándose correctamente
- CORS configurado
- Desplegado en nube con URLs funcionales
- Datos seed precargados
- Variables de entorno (no secrets en codigo)

Calidad General (15%)

- Código limpio y bien organizado
- Nomenclatura consistente
- Documentación clara y reproducible
- Commits incrementales con mensajes descriptivos
- Manejo de errores robusto en ambas capas
- Video demostrativo claro

PUNTOS ADICIONALES (Opcional)

Implementaciones adicionales que suman puntos extra:

1. Testing (+10 puntos)

- Unit tests en Django (models, serializers, calculo IRPC)
- Unit tests en Angular (componentes, servicios, pipes)
- Al menos 1 integration test end-to-end

2. Docker (+8 puntos)

- docker-compose.yml funcional para levantar todo el stack
- Dockerfiles optimizados (multi-stage build para Angular)
- Documentacion de como usar Docker

3. Documentación API Interactiva (+5 puntos)

- Swagger/OpenAPI con drf-spectacular o drf-yasg
- Coleccion de Postman exportada con ejemplos

4. Tiempo Real (+7 puntos)

- WebSockets (Django Channels) para notificaciones de alertas
- Actualizacion automatica del dashboard sin refresh manual

5. Exportación Avanzada (+5 puntos)

- Exportar portafolio a PDF con gráficos incluidos
- Exportar datos a Excel (.xlsx)

6. CI/CD (+5 puntos)

- GitHub Actions pipeline que ejecute tests y haga deploy automatico

---

RESTRICCIONES Y CONSIDERACIONES

1. **NO** usar herramientas RPA comerciales (UiPath, Automation Anywhere) - Backend OBLIGATORIO en Django 4+ con Django REST Framework

2. **NO** usar templates de admin completos tipo AdminLTE como frontend - Frontend OBLIGATORIO en Angular 12+ con Angular Material

3. **NO** copiar código completo de repositorios existentes

4. **SI** permitido usar librerías y paquetes de npm/pip

5. **SI** permitido usar herramientas de AI como apoyo, pero el candidato debe poder explicar cada línea de código en la entrevista

6. Base de datos relacional (PostgreSQL preferido, SQLite aceptado para desarrollo)

7. El código debe ser ejecutable localmente siguiendo las instrucciones del README

FORMATO DE ENTREGA

1. Repositorio GitHub

- Repositorio publico
- Nombre sugerido: datapulse-fullstack-[nombre-apellido]
- Commits incrementales
- Branches si usa Git Flow (main + develop como mínimo)

2. Email de entrega debe incluir:

- Enlace al repositorio GitHub
- URL del frontend desplegado
- URL del backend/API desplegado
- Enlace al video demostrativo (YouTube/Loom/Google Drive)
- Breve descripción de la solución (max. 300 palabras)

ENTREVISTA (Post-entrega)

IMPORTANTE: Posterior a la entrega, se realizará una sesión de defensa técnica de 45-60 minutos donde el candidato deberá:

1. **Explicar decisiones de arquitectura:** Porque organizo el código de esa manera, que alternativas considero.

2. **Hacer modificaciones en vivo:** Se pedirá al candidato realizar un cambio pequeño en el código (ej: agregar un filtro al API, modificar una validación) compartiendo pantalla.

3. **Debugging en vivo:** Se presentará un escenario de error y el candidato deberá diagnosticarlo.

4. **Preguntas conceptuales:** Explicar cómo funciona el ciclo de vida de un request desde el click en Angular hasta la respuesta del backend.

5. **Preguntas de optimización:** Que haría para mejorar el rendimiento si el sistema tuviera 100 países y 1000 usuarios concurrentes.

Esta sesión es obligatoria y representa un filtro crítico. Un desarrollo excelente sin capacidad de explicarlo será evaluado negativamente.

PAISES A MONITOREAR

Código	País	Moneda	Region	Latitud	Longitud
CO	Colombia	COP	ANDINA	4.5709	-74.2973
BR	Brasil	BRL	CONO_SUR	-14.2350	-51.9253
MX	Mexico	MXN	CENTROAMERICA	23.6345	-102.5528
AR	Argentina	ARS	CONO_SUR	-38.4161	-63.6167
CL	Chile	CLP	CONO_SUR	-35.6751	-71.5430
PE	Peru	PEN	ANDINA	-9.1900	-75.0152
EC	Ecuador	USD	ANDINA	-1.8312	-78.1834
UY	Uruguay	UYU	CONO_SUR	-32.5228	-55.7658
PY	Paraguay	PYG	CONO_SUR	-23.4425	-58.4438
PA	Panama	PAB	CENTROAMERICA	8.5380	-80.7821

TIPS

1. **Comience por el backend:** Defina modelos, migraciones y al menos los endpoints CRUD básicos antes de tocar el frontend.

2. **Use Django Admin:** Configure admin.py para poder verificar datos rápidamente durante el desarrollo.

3. **Pruebe las APIs externas primero:** Use Postman o el navegador para entender la estructura de respuesta del World Bank y ExchangeRate-API.

4. **Seed data temprano:** Cree un management command (python manage.py seed_data) para no depender de APIs externas durante el desarrollo.

5. **No deje el despliegue para el final:** Despliegue una versión mínima temprano y vaya actualizando.

6. **Documente mientras desarrolla:** No deje toda la documentación para las últimas horas.

7. **Priorice funcionalidad sobre estética:** Es mejor un CRUD completo con Material básico que un dashboard espectacular sin backend funcional.

8. **Commits frecuentes:** Haga commit después de cada funcionalidad completada.

PREGUNTAS FRECUENTES

P: ¿Puedo usar SQLite en lugar de PostgreSQL?

R: Si, para desarrollo local es aceptable. Para el despliegue en nube, PostgreSQL es preferido, pero SQLite con datos precargados también se acepta.

P: ¿Puedo usar Angular 14, 15 o superior?

R: Si, cualquier versión de Angular 12 o superior es aceptada.

P: ¿Puedo usar una librería CSS adicional como Tailwind?

R: Si, siempre que Angular Material sea la base de componentes UI.

P: ¿Que pasa si una API externa esta caída durante mi desarrollo?

R: Implemente un seed script con datos estáticos y documente el problema. La sincronización desde APIs externas es un bonus, los datos precargados son lo mínimo.

P: Puedo usar ChatGPT/Copilot/Claude como herramienta de apoyo?

R: Si, pero deberá poder explicar y modificar cualquier parte del código en la entrevista. Código que no pueda explicar será evaluado como no propio.

P: ¿El frontend debe funcionar en móvil?

R: No se requiere móvil, pero si debe funcionar en desktop (1920px) y Tablet (768px).

SOPORTE

- **Consultas técnicas:** Máximo 2 preguntas por email
- **Tiempo de respuesta:** Dentro de horario laboral
- **Email contacto:** luis.quinones@mission.com.co

- Solo se responderán consultas sobre clarificación de requerimientos, NO sobre como implementar algo

---

¡Éxito en el desarrollo de la prueba!

Esperamos ver su capacidad de construir una aplicación web completa, funcional y bien estructurada.